

Memory Areas of JVM

1. Stack memory (LIFO)

- stores method call information for each thread
- Stack frame includes: local var, method parameter, return address, reference of heap obj.
- When a method is called, a new frame is added, when a method completes, its frame is removed.
- If too many method calls occur it causes a "StackOverflowError."

2. Heap memory

- stores all objects and class instances.
- shared among all threads.
- two parts: young generation (where new objects created)
 - old generation (where long-lived objects are promoted).
- managed by garbage collector.
- Static var are not stored in heap (from Java 1.8v), but in the Metaspace.
- "OutOfMemoryError", if heap memory is exhausted & GC cannot reclaim memory.

3. Method Area (Metaspace in Java 8+)

- stores class-level information.
- contains class metadata (class name, superclass name), static var, constant pool, method & field info.
- Metaspace which is not a part of heap & grows dynamically using native memory.

4. PC register

- keeps track of current instruction being executed for a thread.
- each thread has its own PC register.
- If executing a native method, the value is undefined.

5. Native method Stack

- supports native methods, such as those written in c/c++ using Java native method Interface.
- works similarly to Java stack but is used for native code execution.

// Stack vs heap memory

Stack memory

- stores method calls, local var and return addresses.
- Memory Allocation LIFO.
- each thread has its own stack.
- faster access (uses stack pointer).
- exists during method execution.
- managed by JVM directly.
- Stack overflow error due to deep recursion or large calls.

Heap memory

- stores objects, arrays and instance var.
- Dynamic memory allocation.
- shared among all threads.
- slower bcoz needs GC and more complex management.
- exists until object is garbage collected.
- JVM & garbage collector.
- OutOfMemory due to heap overflow.

eg: public class Test {

public static void main(String args[]) {

int x = 10; // stored in stack.

String name = "Hello"; // "Hello" (String pool), ref in stack.

Person obj = new Person("Harry"); // obj in stack, person object in heap.

obj.sayHello(); // method call stored in stack.

}
class Person {

String name; // instance var stored in heap

Person(String name) {

{ this.name = name;


```
void sayHello() {
```

```
    sop("Hello "+name);
```

```
}
```

⇒ method sayHello() call → stack frame added, removed after execution.

Garbage collection

→ It is an automatic process that reclaims memory used by objects that are no longer reachable in program. This helps prevent memory leaks and ensure efficient use of memory.

Working

1. Reference checking : JVM checks which objects are still referenced by any var, thread or stack. If an obj has no ref, its marked for collection.
2. Mark and sweep Algorithm
 - mark: gc identifies all reachable objs.
 - sweep: It then cleans up unreferenced objects from the heap.

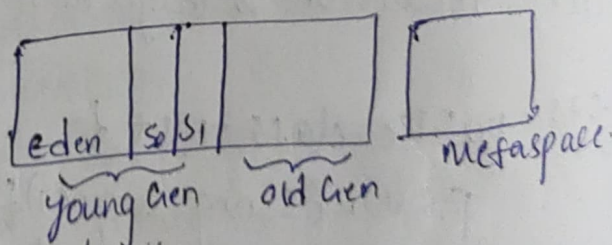
3. phases of GC

→ young generation

- Newly created objs are stored here.
- gc runs frequently here (minor gc).

→ old generation

- Long-living objs get promoted here.
- gc runs less frequently (major gc).



- * when an obj is null, it becomes unreachable - eligible to gc
- * JVM may collect it and call finalize() before deletion.

Different types of Garbage collectors

→ In order to manage memory efficiently, depending on the application's size and performance requirements, Java provides diff types of GC.

- 1) Serial GC
- 2) parallel GC
- 3) concurrent mark sweep
- 4) G1 GC.

Serial GC

- uses a single thread to perform all GC tasks.
- small applications or systems with a single CPU.
- simplicity and low overhead.

parallel GC : (CMS)

- uses multiple threads for both minor and major GC.
- Best for applications where high throughput is more important than low pause times.
- faster than Serial GC due to multithreading.

finalize() : special method in Java that the GC calls before reclaiming memory of an object.

- Allows an obj to perform 'cleanup' operations like closing files or releasing resources before it is destroyed.
- Defined in the Object class, so every class inherits it.
- we can override it in our class, but should not call it manually - it's meant to be called by the JVM.

* Not recommended to use `finalize()` bcoz,

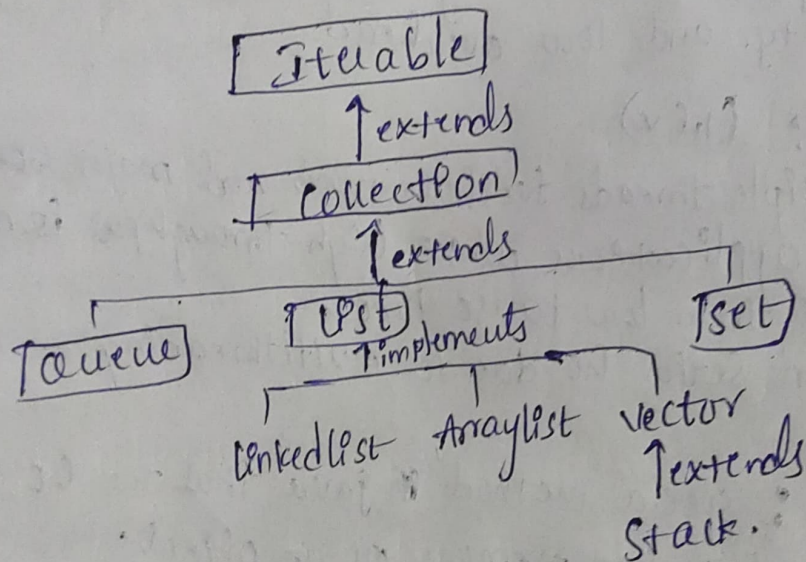
- 1) unpredictable execution - we don't know when or if `finalize()` will be called.
- 2) performance overhead - GC has to track objs with `finalize()`, slowing it down.

Deprecated - in Java 9 & removed in Java 18.
→ Objects can become reachable again inside `finalize()`, causing memory leaks.

Instead of using `finalize()`, use:

- 1) try with resources
- 2) explicit cleanup methods like `close()` for resources such as files etc.

Collection Hierarchy (upto List)



How stack follows LIFO?

Stack is DS, follows LIFO → last element pushed to the stack is the first one removed (popped).

eg:

```
import java.util.Stack;
public class StackDemo {
    public static void main(String[] args) {
```

```
        Stack<String> obj = new Stack<>();
        obj.push("first");
        obj.push("second");
        pop(obj);
```


sop(stack popped: " + obj.pop()); // second
sop(obj.pop()); // first

As shown, the last element pushed ("second") is the first one popped, demonstrating LIFO.

ArrayList

- Dynamic Array
- faster $O(1)$ for get(i);
- slower for middle elements (shifting required).
- less memory usage (data)
- Better for frequent access.

LinkedList

- Doubly LinkedList
- slower $O(n)$ for get(i)
- faster for middle (no shifting)
- more usage of memory (store data + address).
- Better for frequent insert/del.

Stack

- extends Vector
- LIFO
- used as a stack (push, pop, peek)
- limited to stack specific methods.
- more specialized

Vector

- Base class
- random access
- used as a resizable array.
- provides full list methods.
- more general purpose.

ArrayList

- Non Synchronized (not thread safe)
- faster performance
- Grows by 50% of current size.
- part of Java.util (modern collection).
- recommended for single threaded apps.

Vector

- Synchronized (thread safe)
- Slower due to synchronization
- Grows by 100% (doubles in size)
- legacy class (older usage)
- rarely used; prefer Collections, synchronizedList

* To implement a browser's history feature that allows the user to:

- Go back to previous page
- Go forward to next page (after going back).

Java collection to use: "Two stacks"

- Back stack ⇒ when you click back, pop from back and push to forward
- Forward stack ⇒ when you click forward, pop from forward, push to back.

eg: `visit("google.com");`

- push current page to backstack.
- set curr = "google.com"
- clear forwardstack.

`goBack()`

- push current page to forwardstack.
- pop the last page from backstack & set it as

current page.

go forward()

→ push current page to backstack.

→ pop from forwardstack & set it as cur page.

Why stack?

LIFO — the last page visited is the first one to go. back to.

Efficient — push & pop are fast operations.